

LAB 06

Title: Evaluate Postfix Expressions Using C++ with Debugging Techniques.

Objective

To learn how to debug a C++ program using Breakpoints and the Watch Window, and to understand how a postfix (Reverse Polish Notation) expression is evaluated using a stack.

Procedure

1. Complete environment setup.
2. Write the Postfix Expression Evaluation code.
3. Set break points.

Breakpoint Location	Reason
for(char ch : exp)	To observe each character of the postfix expression being processed.
if(isdigit(ch))	To check whether the current character is a digit and should be pushed onto the stack.
else	To detect an operator and prepare to evaluate the top two elements of the stack.

4. Execute with Next Line.
5. Track the changes on Watch.

Code with break-points

```
Postfix.cpp
1  #include <bits/stdc++.h>
2  using namespace std;
3
4  int main()
5  {
6      string exp;
7      cin >> exp;
8
9      stack<int> st;
10
11     for(char ch : exp)
12     {
13         if(isdigit(ch))
14         {
15             st.push(ch - '0');
16         }
17         else
18         {
19             int b = st.top();
20             st.pop();
21
22             int a = st.top();
23             st.pop();
24
25             int result;
26
27             if(ch == '+')
28                 result = a + b;
29             else if(ch == '-')
30                 result = a - b;
31             else if(ch == '*')
32                 result = a * b;
33             else if(ch == '/')
34                 result = a / b;
```

```

35
36     st.push(result);
37 }
38
39
40     cout << "Result = " << st.top();
41
42     return 0;
43 }

```

Output

The screenshot shows a C++ IDE with the following components:

- Project/Classes/Debug:** A tree view on the left showing variables: `ch = 42 '*'`, `b = 2`, `a = 8`, and `result = 16`.
- Postfix.cpp:** The main code file is open, showing a postfix calculator implementation. The code includes a stack-based evaluation of the expression `53+2*`. The current line of execution is `return 0;` at line 42.
- Output Window:** The output of the program is displayed as `53+2*` followed by `Result = 16`.
- Debug Panel:** The bottom of the IDE features a debug toolbar with buttons for `Debug`, `Add watch`, `Next line`, `Continue`, `Next instruction`, `Stop Execution`, `View CPU window`, `Into function`, `Skip function`, and `Into instruction`.